



**Universidad Autónoma de Chiapas,
Campus I,
Escuela de Tecnologías Digitales Aplicadas,**

Práctica: E-commerce “Materiales El Constructor” con arquitectura hexagonal

Documentación técnica — Node.js, Express, Prisma, PostgreSQL y React

Eliann Artemio Grajales Grene

Matrícula 100030369

Ingeniería en Desarrollo y Tecnologías de Software

Taller de Desarrollo 3

Docente: Luis Gutiérrez Alfaro

5.º Semestre Grupo N

Tuxtla Gutiérrez, Chiapas, 24 de septiembre de 2026

Contenido

1. Introducción y objetivos	3
2. Repositorio y tecnologías	4
3. Arquitectura hexagonal	5
4. Estructura del proyecto	7
5. Modelo de datos	8
6. Roles, permisos y seguridad	10
7. Reglas de negocio	11
8. Especificación de la API REST	12
9. Frontend: pantallas de la aplicación	14
10. Diseño visual y auditoría	18
11. Instalación y ejecución	19
12. Pruebas realizadas	20
13. Conclusiones	21
Referencias	22

1. Introducción y objetivos

Para esta práctica desarrollé una tienda en línea de **materiales de construcción** llamada *Materiales El Constructor*. El sistema permite que cualquier persona cree su cuenta y compre de inmediato (cemento, acero, agregados, mampostería, plomería y herramientas), mientras que el administrador controla qué usuarios pueden además gestionar el catálogo de productos o los pedidos de la tienda.

El backend lo construí siguiendo la **arquitectura hexagonal** (puertos y adaptadores): las reglas de negocio viven en un dominio que no depende de Express, Prisma ni bcrypt, y esas tecnologías se conectan en los bordes como adaptadores intercambiables. El frontend es una SPA en React que consume la API REST.

Objetivo general

Implementar un e-commerce funcional con backend en arquitectura hexagonal, persistencia en PostgreSQL mediante Prisma y un frontend en React, aplicando control de acceso por roles y permisos.

Objetivos específicos

- Separar el sistema en capas de dominio, aplicación e infraestructura, con dependencias que apunten siempre hacia el dominio.
- Implementar registro e inicio de sesión con contraseñas cifradas (bcrypt) y tokens JWT.
- Registrar a cada usuario nuevo como **comprador** (el permiso más bajo) sin esperar autorización, y permitir que el administrador le asigne o quite permisos de gestión.
- Garantizar la integridad del inventario: sin stock negativo, aunque dos pedidos lleguen al mismo tiempo.
- Modelar el ciclo de vida de un pedido (pendiente, confirmado, enviado, entregado, cancelado) con transiciones controladas.
- Diseñar una interfaz accesible y responsiva, y auditarla con herramientas de diseño de UI/UX.

Alcance

Módulo	Funcionalidad
Cuentas	Registro con inicio de sesión inmediato, login, sesión con JWT, cierre de sesión.
Tienda	Catálogo con búsqueda, filtro por categoría, precio por unidad de venta, carrito y confirmación de pedido.
Mis pedidos	Historial de compras del usuario con el estado de cada entrega.
Productos	Alta, edición y baja de materiales (permiso <code>PRODUCTS</code>).
Pedidos	Vista de todos los pedidos y cambio de estado (permiso <code>ORDERS</code>).
Usuarios	Asignar o quitar permisos y revocar o reactivar cuentas (solo administrador).

2. Repositorio y tecnologías

Repositorio en GitHub: <https://github.com/Rorre12/ecommerce-practica>

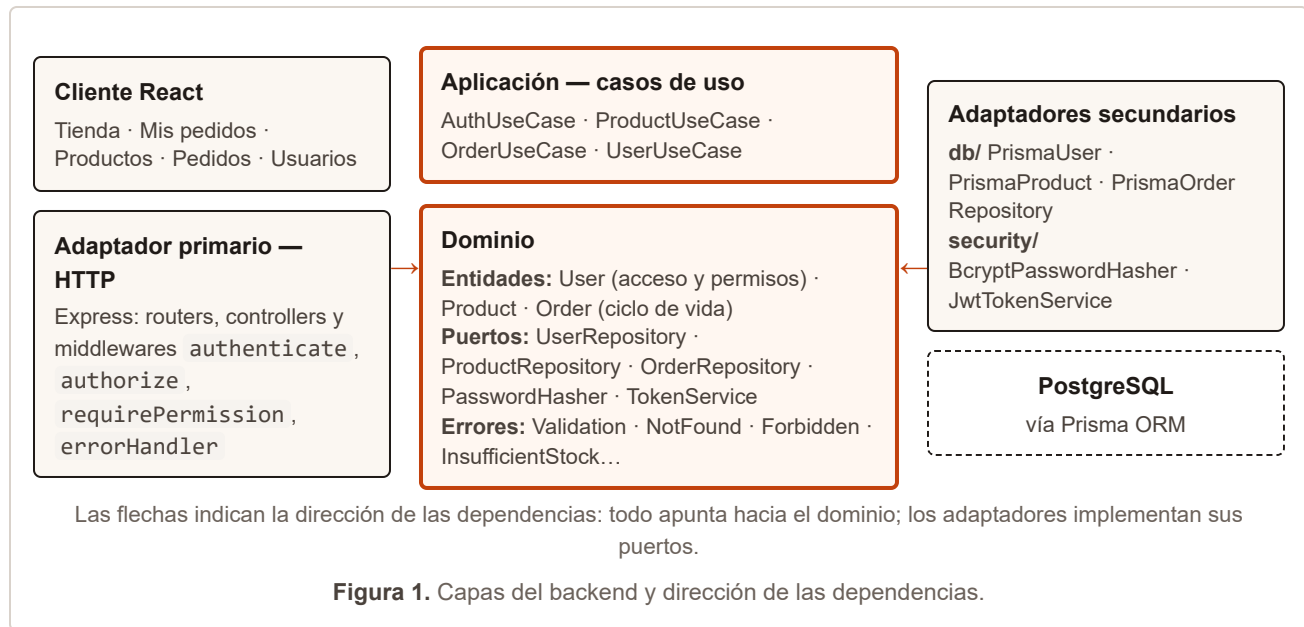
Clonar: `git clone https://github.com/Rorre12/ecommerce-practica.git`

El repositorio contiene el código del backend (`server/`), el frontend (`client/`), la documentación técnica en Markdown (`docs/` , con capturas en `docs/img/`) y este documento en PDF.

Capa	Tecnología	Uso en el proyecto
Backend	Node.js 18+ y Express 4	Servidor HTTP, rutas, controladores y middlewares
	Prisma ORM 5	Acceso a datos, migraciones y seed
	PostgreSQL	Base de datos relacional
	bcryptjs	Hash de contraseñas (adaptador del puerto <code>PasswordHasher</code>)
	jsonwebtoken	Tokens de sesión (adaptador del puerto <code>TokenService</code>)
Frontend	React 18 + Vite 5	SPA y servidor de desarrollo con proxy a la API
	lucide-react	Iconos SVG de la interfaz
	CSS con tokens OKLCH	Sistema visual con modo claro y oscuro
	Big Shoulders Display + IBM Plex Sans	Tipografías (Google Fonts)
Herramientas	Git y GitHub	Control de versiones y publicación
	Playwright + Chrome	Pruebas de la interfaz en el navegador y capturas

3. Arquitectura hexagonal

La arquitectura hexagonal organiza la aplicación alrededor del **dominio**. El dominio define *puertos* (interfaces) para todo lo que necesita del exterior, como guardar un usuario o cifrar una contraseña, y la infraestructura aporta *adaptadores* que implementan esos puertos con tecnologías concretas. Así, cambiar Prisma por otro ORM o bcrypt por argon2 solo afecta al adaptador correspondiente.



Capas

Capa	Contenido y responsabilidad
Dominio src/domain	JavaScript puro, sin dependencias externas. <code>User</code> valida credenciales y decide el acceso (<code>can</code> , <code>assertCanAccess</code> , <code>review</code>); <code>Product</code> valida precio, stock, unidad y categoría (<code>assertStock</code>); <code>Order</code> congela precios, calcula totales y controla el ciclo de vida (<code>assertCanChangeTo</code> , <code>nextStatuses</code>). Aquí están también los errores de dominio y los puertos.
Aplicación src/application	Casos de uso que orquestan entidades y puertos: <code>AuthUseCase</code> (<code>register</code> , <code>login</code> , <code>me</code> , <code>getActiveUser</code>), <code>ProductUseCase</code> (<code>CRUD</code>), <code>OrderUseCase</code> (<code>create</code> , <code>list</code> , <code>getById</code> , <code>changeStatus</code>) y <code>UserUseCase</code> (<code>list</code> , <code>review</code>). Reciben sus dependencias por constructor.
Infraestructura src/infrastructure	Adaptador primario HTTP (Express) y adaptadores secundarios de persistencia (Prisma) y seguridad (bcryptjs, JWT). El <code>errorHandler</code> traduce el <code>code</code> de cada error de dominio a un estado HTTP.
Raíz de composición src/index.js	Único punto que conoce las clases concretas: crea los adaptadores, los inyecta en los casos de uso y arranca el servidor.

Flujo de una petición: crear un pedido

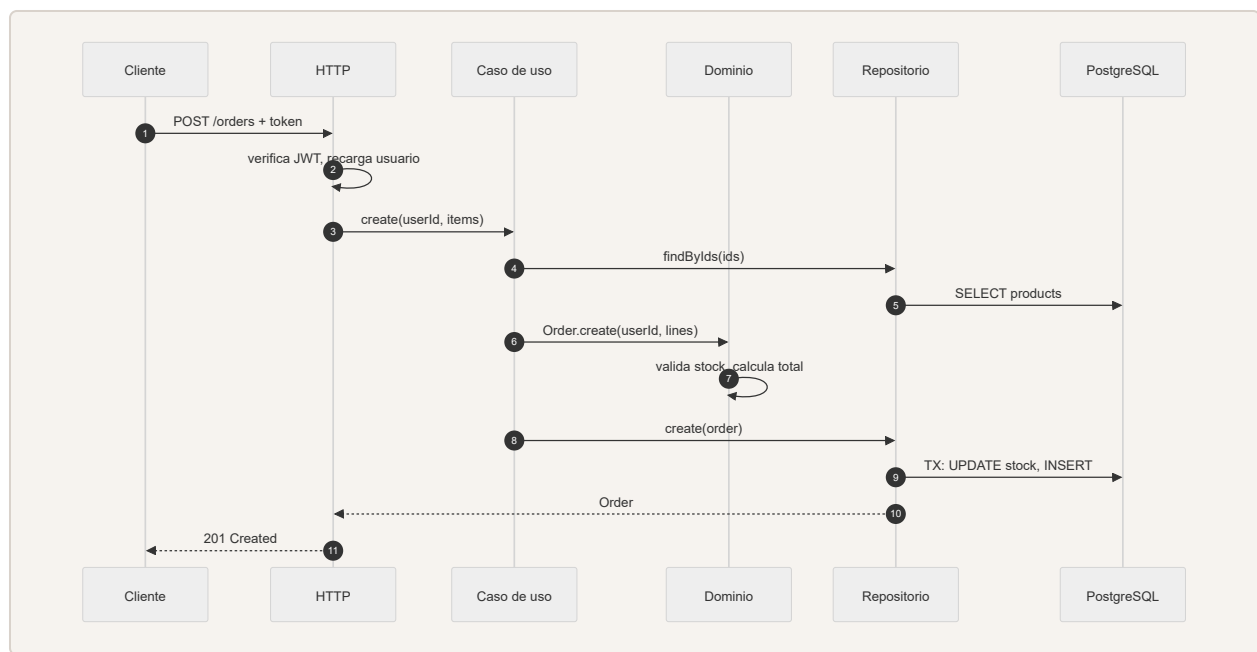


Figura 2. Recorrido de `POST /api/orders` a través de las capas.

4. Estructura del proyecto

```
ecommerce-practica/
├── README.md · docker-compose.yml · Documentacion-Ecommerce-Practica.pdf
├── docs/                                # arquitectura, api, base-de-datos, frontend, instalacion,
                                        # auditoria-diseno + capturas en docs/img
├── server/
│   ├── prisma/
│   │   ├── schema.prisma                # User, Product, Order, OrderItem + enums
│   │   ├── migrations/                  # init · roles_permisos_materiales · registro_directo
│   │   └── seed.js                      # administrador + 12 materiales de construcción
│   └── src/
│       ├── index.js                    # raíz de composición
│       ├── domain/
│       │   ├── errors.js
│       │   ├── entities/                # User.js · Product.js · Order.js
│       │   └── ports/                   # UserRepository · ProductRepository · OrderRepository
│       │                                       # PasswordHasher · TokenService
│       ├── application/                 # AuthUseCase · ProductUseCase · OrderUseCase · UserUseCase
│       └── infrastructure/adapters/
│           ├── http/                   # app.js · routes/ · controllers/ · middlewares/
│           ├── db/                     # prismaClient · PrismaUser/Product/OrderRepository
│           └── security/                # BcryptPasswordHasher · JwtTokenService
└── client/
    ├── index.html · public/favicon.svg · vite.config.js (proxy /api)
    └── src/
        ├── App.jsx · api.js · format.js · styles.css · main.jsx
        ├── components/ui.jsx           # logotipo, avisos, estados, iconos de categoría
        └── modules/                    # auth · store · orders · products · users
```

Reglas de dependencia

- `domain` no importa nada de `application` ni de `infrastructure`.
- `application` importa solo del dominio.
- `infrastructure` importa del dominio para implementar los puertos.

5. Modelo de datos

La base de datos es PostgreSQL, administrada con Prisma. El esquema está en `server/prisma/schema.prisma`.

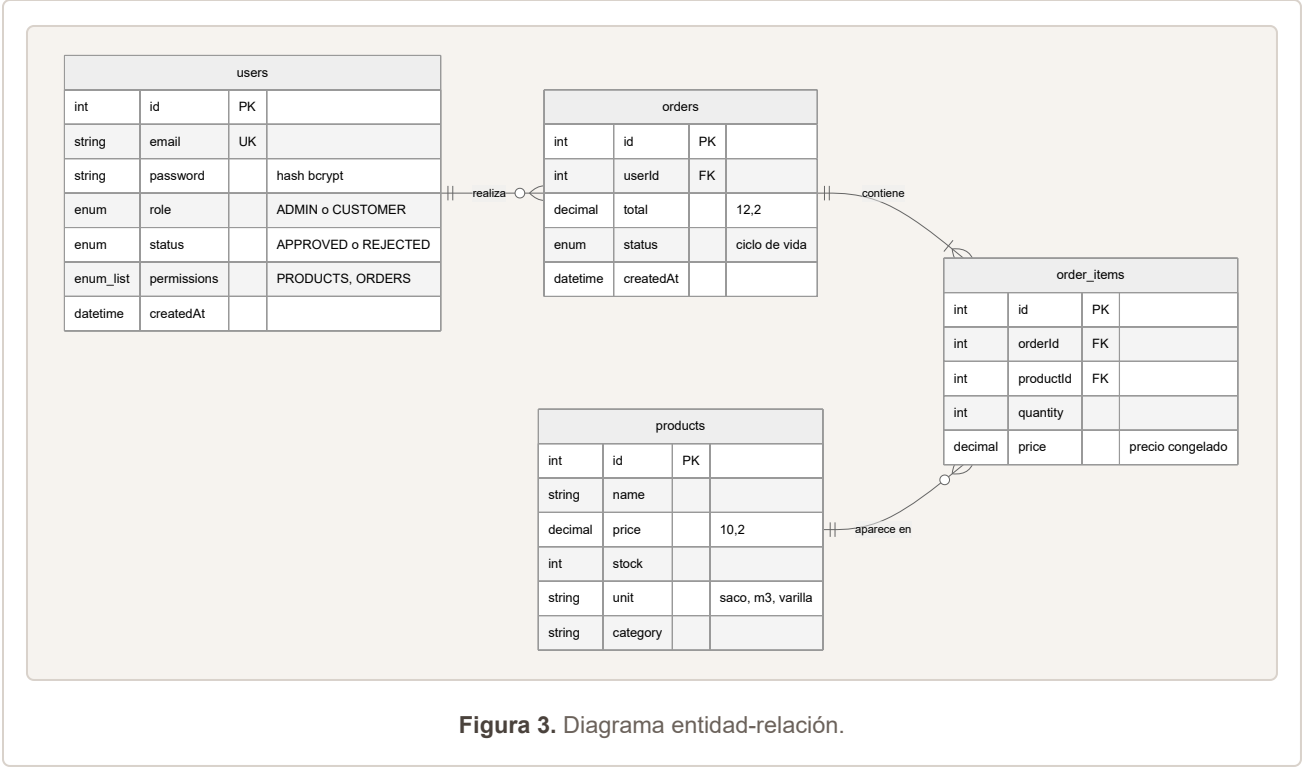


Figura 3. Diagrama entidad-relación.

Enum	Valores	Uso
Role	ADMIN, CUSTOMER	Tipo de cuenta
UserStatus	APPROVED, REJECTED, PENDING	Acceso. Las cuentas nacen APPROVED; REJECTED es acceso revocado; PENDING se conserva por compatibilidad
Permission	PRODUCTS, ORDERS	Pantallas de gestión que habilita el administrador
OrderStatus	PENDING, CONFIRMED, SHIPPED, DELIVERED, CANCELLED	Ciclo de vida del pedido

Decisiones de diseño

- Precio congelado:** `order_items.price` guarda el precio vigente al comprar, así el historial no cambia si después se edita el producto.
- Decimal para dinero:** `Decimal(10,2)` en la base de datos y cálculo del total en centavos en el dominio, para evitar errores de coma flotante.
- Integridad referencial:** un producto que ya forma parte de pedidos no se puede eliminar (`onDelete: Restrict`).

Migraciones

Migración	Cambios
<code>init</code>	Tablas <code>users</code> , <code>products</code> , <code>orders</code> y <code>order_items</code> .
<code>roles_permisos_materiales</code>	Enums de estado y permisos; columnas de estado, permisos, unidad y categoría; los usuarios existentes conservan el acceso.
<code>registro_directo</code>	El valor por defecto de <code>users.status</code> pasa a <code>APPROVED</code> (registro sin espera).

6. Roles, permisos y seguridad

Al crear su cuenta, cada persona entra de inmediato como **comprador**, el nivel más bajo: puede usar la Tienda y consultar sus pedidos. A partir de ahí, el administrador decide desde el panel de Usuarios qué pantallas de gestión le habilita, o si le revoca el acceso.

Perfil	Cómo se obtiene	Pantallas
Comprador	Al registrarse, sin esperar autorización	Tienda · Mis pedidos
Comprador + <code>PRODUCTS</code>	Lo asigna el administrador	+ Productos (crear, editar, eliminar)
Comprador + <code>ORDERS</code>	Lo asigna el administrador	+ Pedidos (ver todos y cambiar estados)
Administrador	Lo crea el seed	Tienda · Productos · Pedidos · Usuarios

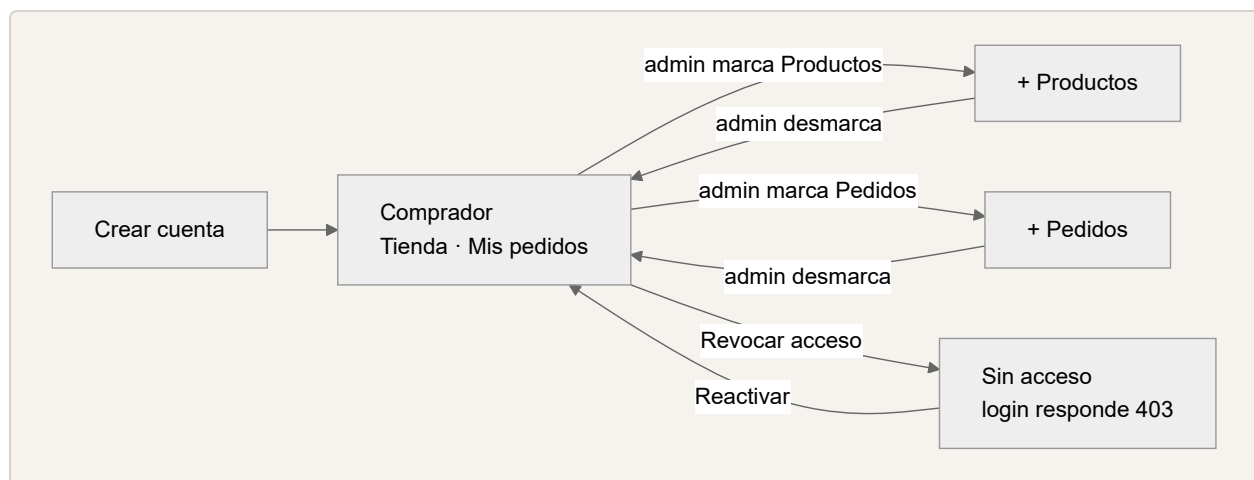


Figura 4. Ciclo de acceso de una cuenta.

Cómo se aplican las reglas

- **En el backend, no solo en la interfaz.** El middleware `authenticate` verifica el JWT y **recarga al usuario desde la base de datos en cada petición**. Por eso un cambio de permisos o una revocación aplica en la siguiente acción del usuario, sin que tenga que cerrar sesión.
- Las rutas se protegen con `requirePermission('PRODUCTS' | 'ORDERS')` y `authorize('ADMIN')`. Si alguien llama a la API directamente sin el permiso, recibe `403 FORBIDDEN`.
- La decisión la toma siempre la entidad `User` del dominio: `can(permiso)` (el admin los tiene todos), `assertCanAccess()` y `review()`, que impide modificar el acceso de un administrador.
- Las contraseñas se guardan con hash `bcrypt` y nunca se devuelven; `toPublic()` las omite en todas las respuestas.
- Los secretos (`DATABASE_URL`, `JWT_SECRET`) viven en `server/.env`, que está excluido del repositorio por `.gitignore`.

7. Reglas de negocio

Inventario sin stock negativo

1. El dominio (`Order.create` → `Product.assertStock`) valida las cantidades y el stock disponible.
2. El repositorio descuenta el stock **dentro de una transacción** con `UPDATE ... SET stock = stock - cantidad WHERE id = ... AND stock >= cantidad` . Si otro pedido se llevó el stock un instante antes, la actualización no afecta ninguna fila, se lanza `INSUFFICIENT_STOCK` y se revierte todo el pedido.
3. Si el mismo producto viene varias veces en el pedido, las cantidades se suman antes de validar.

Ciclo de vida del pedido

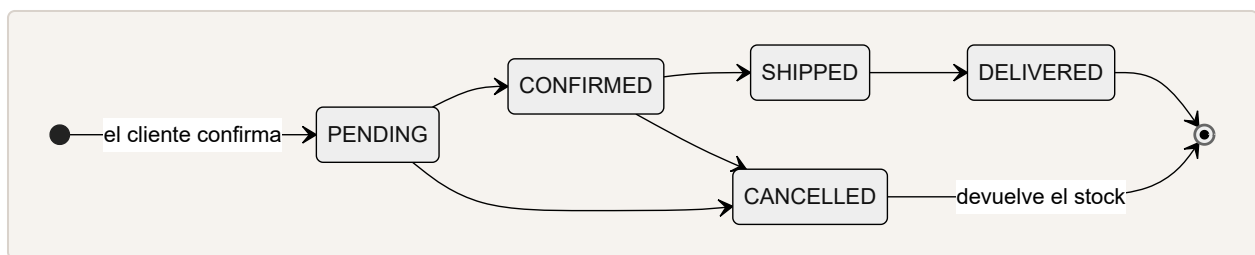


Figura 5. Transiciones permitidas. Cualquier otra se rechaza con `400` .

- La tabla de transiciones vive en la entidad `Order` . La API devuelve en cada pedido el campo `nextStatuses` , y el frontend solo muestra los botones de acciones válidas.
- Al **cancelar**, el stock de cada línea se devuelve al inventario en la misma transacción.
- El cambio de estado está condicionado al estado leído (`WHERE status = estadoAnterior`): si dos personas modifican el mismo pedido a la vez, la segunda recibe `409` en lugar de sobrescribir el cambio.

8. Especificación de la API REST

Base URL: `http://localhost:4000/api` · **Autenticación:** `Authorization: Bearer <token>` · **Formato de error:** `{ "error": { "code", "message", "details" } }`

Método	Ruta	Acceso	Descripción
GET	<code>/health</code>	Público	Estado de la API
POST	<code>/auth/register</code>	Público	Crea un comprador y devuelve <code>{ user, token }</code>
POST	<code>/auth/login</code>	Público	<code>{ user, token }</code> ; 403 si el acceso fue revocado
GET	<code>/auth/me</code>	Sesión	Usuario con permisos vigentes
GET	<code>/products</code> , <code>/products/:id</code>	Público	Catálogo
POST · PUT · DELETE	<code>/products[:id]</code>	PRODUCTS	Gestión del catálogo
POST	<code>/orders</code>	Sesión	Crear pedido (descuenta stock)
GET	<code>/orders</code>	Sesión	Pedidos propios
GET	<code>/orders?scope=all</code>	ORDERS	Todos los pedidos
GET	<code>/orders/:id</code>	Dueño u ORDERS	Detalle de un pedido
PATCH	<code>/orders/:id/status</code>	ORDERS	Avanzar o cancelar un pedido
GET	<code>/users[?status=]</code>	Admin	Lista de usuarios
PATCH	<code>/users/:id</code>	Admin	Cambiar <code>permissions</code> y/o <code>status</code>

Códigos de error

Código de dominio	HTTP	Cuándo ocurre
VALIDATION_ERROR	400	Datos inválidos, permiso inexistente o transición de estado no permitida
UNAUTHORIZED	401	Sin token, token inválido o expirado, o credenciales incorrectas
FORBIDDEN	403	Falta el permiso o el acceso fue revocado
NOT_FOUND	404	Recurso inexistente, o pedido de otro usuario
CONFLICT	409	Email ya registrado, producto con pedidos o pedido modificado por otro usuario
INSUFFICIENT_STOCK	409	La cantidad pedida supera el stock

Ejemplos

```
// POST /api/auth/register          → 201 Created
{ "email": "constructora.norte@obra.com", "password": "secreto1" }
{ "user": { "id": 21, "email": "constructora.norte@obra.com", "role": "CUSTOMER",
            "status": "APPROVED", "permissions": [] },
  "token": "eyJhbGciOiJIUzI1NiIs..." }

// PATCH /api/users/21 (admin)      → 200 OK
{ "permissions": ["ORDERS"] }

// PATCH /api/orders/15/status (ORDERS) → 200 OK
{ "status": "SHIPPED" }
{ "id": 15, "status": "SHIPPED", "nextStatuses": ["DELIVERED"], "total": 759, ... }

// PATCH /api/orders/15/status con { "status": "PENDING" } → 400
{ "error": { "code": "VALIDATION_ERROR",
            "message": "No se puede cambiar un pedido de SHIPPED a PENDING" } }
```

La referencia completa, con los cuerpos de cada endpoint y ejemplos `curl`, está en `docs/api.md` del repositorio.

9. Frontend: pantallas de la aplicación

La navegación lateral se arma a partir de los permisos del usuario: cada pantalla declara quién puede verla, y al cargar la aplicación se consulta `/api/auth/me` para reflejar los permisos vigentes. Las capturas siguientes se tomaron de la aplicación en ejecución.

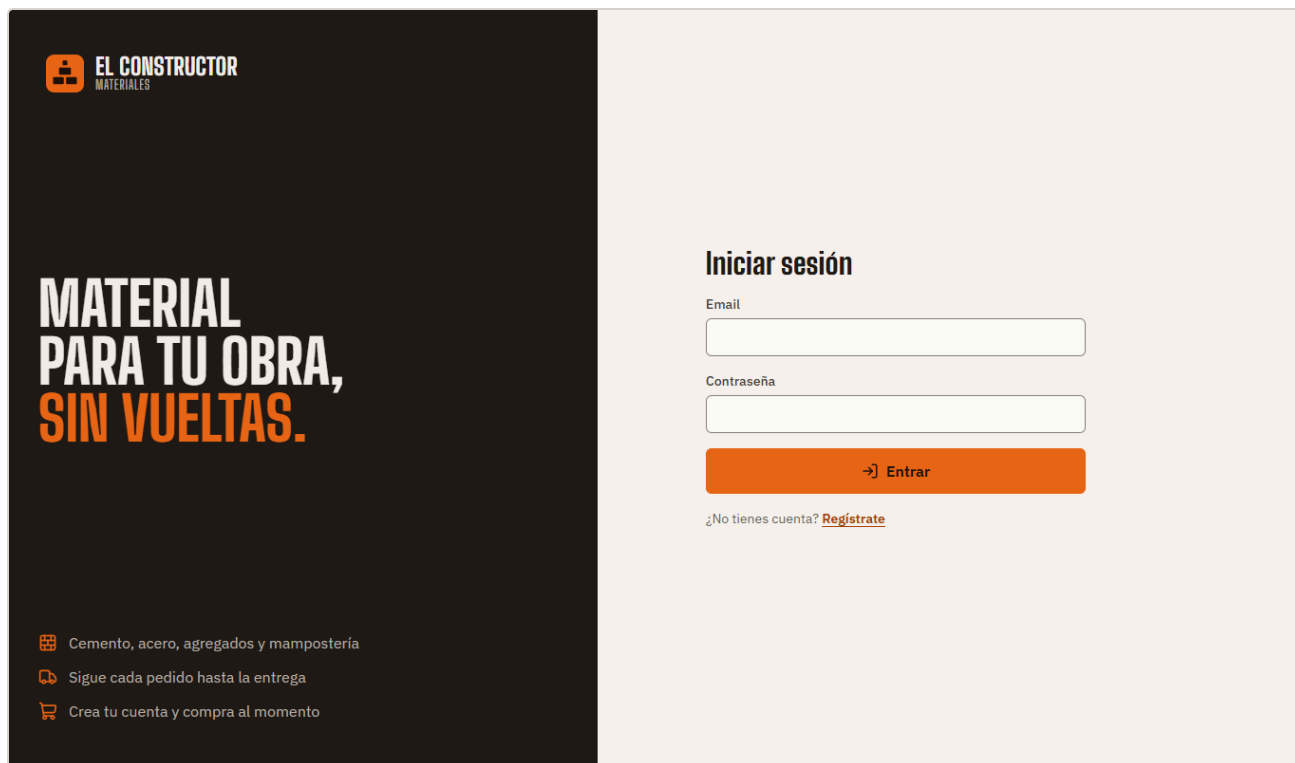


Figura 6. Inicio de sesión.


EL CONSTRUCTOR
MATERIALES

MATERIAL PARA TU OBRA, SIN VUELTAS.

-  Cemento, acero, agregados y mampostería
-  Sigue cada pedido hasta la entrega
-  Crea tu cuenta y compra al momento

Crear cuenta

Podrás comprar en cuanto termines el registro.

Email

Contraseña

Entre 6 y 72 caracteres.

 **Crear cuenta**

¿Ya tienes cuenta? [Inicia sesión](#)

Figura 7. Crear cuenta: al terminar, la persona entra directo a la tienda como comprador.


EL CONSTRUCTOR
MATERIALES

Tienda

Productos

Pedidos

Usuarios

ADMINISTRADOR
admin@tienda.com

[Salir](#)

Tienda

Materiales disponibles para entrega. Precios por unidad de venta.

Buscar material

Todas
Acero
Agregados
Cementos y morteros
Herramientas

Mampostería
Plomería

Acero

Alambre recocido

2,10 US\$ / kg

150 kg disponibles

- 1 + **Agregar**

Acero

Varilla corrugada 1/2" x 12 m

13,40 US\$ / varilla

298 varilla disponibles

- 1 + **Agregar**

Acero

Varilla corrugada 3/8" x 12 m

7,90 US\$ / varilla

499 varilla disponibles

- 1 + **Agregar**

Agregados

Arena de río

28,00 US\$ / m³

40 m³ disponibles

- 1 + **Agregar**

Agregados

Grava 3/4"

Calidadada 25 kg

- 1 + **Agregar**

Cementos y morteros

Cemento gris 50 kg

25 saco x 9,50 US\$

237,50 US\$

Quitar

Cementos y morteros

Varilla corrugada 3/8" x 12 m

40 varilla x 7,90 US\$

316,00 US\$

Quitar

Carrito 2

Total 553,50 US\$

Confirmar pedido

Figura 8. Tienda: búsqueda, filtros por categoría, precio por unidad de venta, selector de cantidad y carrito.

EL CONSTRUCTOR
MATERIALES

Tienda

Mis pedidos

CLIENTE
constructora.norte@obra.com

Salir

Mis pedidos

Historial de tus compras y el estado de cada entrega.

Estado
Todos

N.º	FECHA	MATERIALES	TOTAL	ESTADO
16	24 sept 2026, 18:25	500 pieza × Block de concreto 15x20x40 (0,95 US\$) 3 m² × Arena de río (28,00 US\$)	559,00 US\$	Pendiente
15	24 sept 2026, 18:25	30 saco × Cemento gris 50 kg (9,50 US\$) 60 varilla × Varilla corrugada 3/8" x 12 m (7,90 US\$)	759,00 US\$	Enviado

Figura 9. Vista de un comprador: solo ve Tienda y Mis pedidos, con el estado de cada entrega.

EL CONSTRUCTOR
MATERIALES

Tienda

Productos

Pedidos

Usuarios

ADMINISTRADOR
admin@tienda.com

Salir

Usuarios

Cada cuenta nueva entra como comprador. Marca las pantallas extra que puede usar cada persona o revócale el acceso.

Usuarios registrados

EMAIL	REGISTRO	ESTADO	PERMISOS	ACCIONES
constructora.norte@obra.com	24 sept 2026, 18:25	Activo	☒ Tienda ☐ Productos ☐ Pedidos	Revocar acceso
jorge@gmail.com	24 sept 2026, 17:54	Activo	☒ Tienda ☒ Productos ☒ Pedidos	Revocar acceso
pablo@gmail.com	24 sept 2026, 17:53	Activo	☒ Tienda ☐ Productos ☒ Pedidos	Revocar acceso
elianngrajales@gmail.com	24 sept 2026, 17:21	Activo	☒ Tienda ☒ Productos ☐ Pedidos	Revocar acceso
admin@tienda.com Admin	24 sept 2026, 17:21	Activo	Acesso total	

Figura 10. Panel de Usuarios del administrador: casillas de permisos por persona y botones para revocar o reactivar el acceso.

- Tienda
- Productos
- Pedidos**
- Usuarios

ADMINISTRADOR
admin@tienda.com
[Salir](#)

Pedidos

Todos los pedidos de la tienda. Avanza cada uno por su ciclo: confirmado, enviado y entregado.

Estado Todos

N.º	FECHA	CLIENTE	MATERIALES	TOTAL	ESTADO	ACCIONES
16	24 sept 2026, 18:25	constructora.norte@obra.com	500 pieza x Block de concreto 15x20x40 (0,95 US\$) 3 m³ x Arena de río (28,00 US\$)	559,00 US\$	Pendiente	Confirmar Cancelar
15	24 sept 2026, 18:25	constructora.norte@obra.com	30 saco x Cemento gris 50 kg (9,50 US\$) 60 varilla x Varilla corrugada 3/8" x 12 m (7,90 US\$)	759,00 US\$	Enviado	Entregar
14	24 sept 2026, 18:09	jorge@gmail.com	1 kg x Alambre recocido (2,10 US\$)	2,10 US\$	Enviado	Entregar
7	24 sept 2026, 17:48	elianngrajales@gmail.com	1 varilla x Varilla corrugada 3/8" x 12 m (7,90 US\$) 1 varilla x Varilla corrugada 1/2" x 12 m (13,40 US\$)	21,30 US\$	Entregado	Finalizado
6	24 sept 2026, 17:47	admin@tienda.com	1 varilla x Varilla corrugada 1/2" x 12 m (13,40 US\$)	13,40 US\$	Confirmado	Enviar Cancelar

Figura 11. Gestión de pedidos: solo aparecen las acciones válidas para el estado de cada pedido.

- Tienda
- Productos**
- Pedidos
- Usuarios

ADMINISTRADOR
admin@tienda.com
[Salir](#)

Productos

Catálogo de la tienda: precios, existencias y unidad de venta.

+ Nuevo producto

Nombre

Precio (US\$)

Stock

Unidad de venta

Categoría

saco, varilla, m³, pieza...

Cementos, Acero...

+ Crear producto

ID	NOMBRE	CATEGORÍA	PRECIO	STOCK	ACCIONES
12	Alambre recocido	Acero	2,10 US\$ / kg	150	Editar Eliminar
11	Varilla corrugada 1/2" x 12 m	Acero	13,40 US\$ / varilla	298	Editar Eliminar
10	Varilla corrugada 3/8" x 12 m	Acero	7,90 US\$ / varilla	499	Editar Eliminar
8	Arena de río	Agregados	28,00 US\$ / m³	40	Editar Eliminar

Figura 12. Gestión del catálogo: alta y edición con unidad de venta y categoría.

10. Diseño visual y auditoría

Para el diseño tomé como referencia dos herramientas: **UI UX Pro Max** (uupm.cc), que recomienda estilos, paletas y tipografías según el tipo de producto e incluye un checklist de accesibilidad, y **Hallmark** (github.com/nutlope/hallmark), que audita una interfaz contra una lista de anti-patrones propios de los diseños genéricos. El detalle de cada hallazgo, con su ubicación y corrección, está en docs/auditoria-diseno.md del repositorio.

Sistema visual

- **Dirección:** para “Construction/Architecture”, UI UX Pro Max recomienda un estilo suizo e industrial: gris industrial, naranja de seguridad y azul plano.
- **Color:** tokens en OKLCH, neutros cálidos, sin blanco ni negro puros y un solo acento naranja. Modo oscuro automático.
- **Tipografía:** Big Shoulders Display (condensada industrial) para títulos y precios, e IBM Plex Sans para el texto. Las cifras de las tablas son tabulares, para que queden alineadas.
- **Iconos:** una sola librería (Lucide) y un logotipo SVG propio.

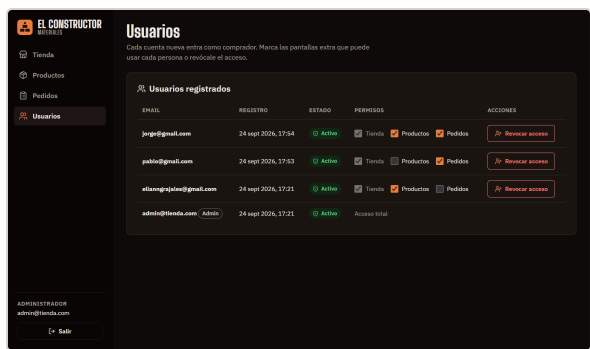


Figura 13. Modo oscuro.



Figura 14. Móvil (390 px).

Resultado de la auditoría

Criterio	Antes	Después (medido en el navegador)
Hallmark: anti-patrones	4 críticos · 5 mayores · 4 menores	0 críticos · 0 mayores · 4 menores (justificados)
Contraste de texto (mín. 4.5:1)	3.19:1 en los botones	≥ 5.4:1 en modo claro y oscuro
Bordes de controles (mín. 3:1)	No medido	3.36 – 3.83:1
Áreas táctiles (mín. 44 px)	~34 px	0 controles por debajo de 44 px en pantallas táctiles
Scroll horizontal	—	0 px de 375 a 2560 px de ancho
Foco con teclado	Sin estilo en botones	Anillo visible e instantáneo

11. Instalación y ejecución

Requisitos: Node.js 18 o superior, PostgreSQL 14 o superior (o Docker) y Git.

```
# 1) Clonar el repositorio
git clone https://github.com/Rorre12/ecommerce-practica.git
cd ecommerce-practica

# 2) Base de datos (opcional si PostgreSQL ya está instalado)
docker compose up -d

# 3) Backend
cd server
copy .env.example .env          # ajustar DATABASE_URL y JWT_SECRET
npm install
npx prisma migrate dev          # aplica las 3 migraciones
npm run seed                    # admin@tienda.com / Admin123! + 12 materiales
npm run dev                     # http://localhost:4000

# 4) Frontend (otra terminal)
cd client
npm install
npm run dev                     # http://localhost:5173
```

Variable (server/.env)	Ejemplo	Descripción
DATABASE_URL	postgresql://postgres:postgres@localhost:5432/ecommerce	Conexión a PostgreSQL
PORT	4000	Puerto de la API
CORS_ORIGIN	http://localhost:5173	Orígenes permitidos
JWT_SECRET	cadena larga aleatoria	Firma de los tokens (obligatoria)
JWT_EXPIRES_IN	8h	Duración del token
ADMIN_EMAIL / ADMIN_PASSWORD	admin@tienda.com / Admin123!	Administrador que crea el seed

Al abrir **http://localhost:5173** se puede entrar como administrador con las credenciales del seed, o crear una cuenta nueva, que entra de inmediato como comprador.

12. Pruebas realizadas

Probé el sistema en dos niveles: contra la API con `curl`, y en el navegador con Chrome controlado por Playwright, recorriendo la interfaz como lo haría un usuario. Al final de cada prueba borré los datos generados.

Pruebas de la API

#	Caso	Esperado	Resultado
1	Registro de una cuenta nueva	201 con token; APPROVED sin permisos	Correcto
2	Login inmediato después del registro	200 con token	Correcto
3	Comprador crea un pedido	201; descuenta stock	Correcto
4	Comprador edita un producto	403	Correcto
5	Comprador consulta todos los pedidos	403	Correcto
6	Admin asigna <code>PRODUCTS</code> ; el usuario edita (mismo token)	200	Correcto
7	Admin quita los permisos; el usuario intenta editar	403	Correcto
8	Admin revoca el acceso; el usuario intenta entrar	403 "acceso revocado"	Correcto
9	Registro con un email duplicado	409	Correcto
10	Transición inválida <code>CONFIRMED</code> → <code>DELIVERED</code>	400	Correcto
11	Cancelar un pedido devuelve el stock	Stock 198 → 201	Correcto
12	Intentar modificar el acceso del administrador	400	Correcto
13	Asignar un permiso inexistente (<code>USERS</code>)	400	Correcto

Pruebas en el navegador

Flujo	Resultado
Crear una cuenta desde la pantalla → entra a la tienda con "Tienda" y "Mis pedidos"	Correcto
El comprador agrega materiales al carrito y confirma el pedido	Correcto
El admin marca Productos y Pedidos; al volver a entrar, el usuario ve 4 pestañas	Correcto
Un usuario con permiso de Pedidos confirma un pedido desde la gestión	Correcto
Sin errores en la consola del navegador	Correcto
Anchos 375, 768, 1024, 1366, 1440, 1920 y 2560 px sin scroll horizontal	Correcto

13. Conclusiones

La arquitectura hexagonal demostró su utilidad a lo largo de la práctica. Cuando cambió el requisito de registro (primero con aprobación previa del administrador y después con acceso inmediato como comprador), el cambio se concentró en la entidad `User` y en `AuthUseCase`, sin tocar Express ni los repositorios. Lo mismo ocurrió al agregar permisos y estados de pedido: las reglas quedaron en el dominio y los adaptadores solo las consultan.

También comprobé la importancia de validar los permisos en el servidor. Ocultar una pestaña en React no impide que alguien llame a la API directamente; por eso cada ruta protegida verifica el permiso, y el usuario se recarga desde la base de datos en cada petición para que los cambios del administrador apliquen de inmediato.

En cuanto a la integridad de los datos, el descuento de stock condicionado dentro de una transacción evita inventarios negativos con pedidos simultáneos, y el precio congelado en cada línea conserva un historial confiable.

Por último, auditar la interfaz con herramientas externas y medir en el navegador (contraste, áreas táctiles, desbordes) reveló problemas que no se veían leyendo el código, como una página que desbordaba en móvil o reglas de CSS que no se aplicaban por el orden de la cascada. La interfaz final es accesible, responsiva y coherente con el giro de la tienda.

Referencias

- Cockburn, A. (2005). *Hexagonal architecture*. <https://alistair.cockburn.us/hexagonal-architecture/>
- Express. (s. f.). *Express — Node.js web application framework*. <https://expressjs.com/>
- Prisma. (s. f.). *Prisma ORM documentation*. <https://www.prisma.io/docs>
- PostgreSQL Global Development Group. (s. f.). *PostgreSQL documentation*. <https://www.postgresql.org/docs/>
- React. (s. f.). *React documentation*. <https://react.dev/>
- Auth0. (s. f.). *Introduction to JSON Web Tokens*. <https://jwt.io/introduction>
- W3C. (2023). *Web Content Accessibility Guidelines (WCAG) 2.2*. <https://www.w3.org/TR/WCAG22/>
- UI UX Pro Max. (s. f.). *Design intelligence skill*. <https://uupm.cc/>
- Nutlope. (s. f.). *Hallmark: anti-AI design skill*. <https://github.com/nutlope/hallmark>
- Lucide. (s. f.). *Lucide icons*. <https://lucide.dev/>

Código fuente: <https://github.com/Rorre12/ecommerce-practica>